Data Structures & Algorithms

Arrays, linked lists, sorting, recursion, DP, trees/graphs, and real interview coding problems

Compiled from real interview experiences (GeeksforGeeks, GrowthGear, Cipher Schools, Zalon, CREATE Bytes) + curated additions

---

Topics covered: arrays & strings · linked lists · sorting/searching · recursion & backtracking · dynamic programming · trees & graphs · sliding window · heaps/tries/LRU cache · real fraud-detection style problems
Difficulty split: Easy → Medium → Hard, moving from brute-force to optimized approaches

Format: Every section is split into **Easy** → **Medium** → **Hard**, and spans theoretical concepts, output-based/tricky code, and practical / machine-coding style problems.

EASY LEVEL

Arrays & Strings

1. Given an array of integers, find the pair with the maximum sum.

2. Given a sorted/unsorted array, implement the classic Two Sum problem (return indices of the two numbers that add up to a target).

3. Reverse a string / array in place without using built-in reverse methods.

4. Check whether a given string is a palindrome.

5. Find the first non-repeating character in a string.

Sorting Basics

6. Implement Bubble Sort from scratch (numerical ascending) on a string array converted to numbers.

```
let arr = ["12", "2", "22", "56", "2"];
// Bubble sort ascending by numeric value
```

7. What is the time and space complexity of Bubble Sort, Selection Sort, and Insertion Sort?

8. Sort an array containing only 3 distinct values (Dutch National Flag style) — e.g. an array of 'a', 'b', 'c' — in O(n) time.

```
var arr = ['c','a','b','a','b','b','b','c','c','c','c'];
// sort ascending and descending in-place
```

Basic Math / Number Theory

9. Check whether a given number is prime — write the brute-force O(n) approach first.

10. Print all prime numbers in a given range using the brute-force approach.

11. Find the factorial of a number using recursion and iteration.

12. Check if a number is a palindrome (numeric palindrome, e.g. 121).

Linked List Basics

13. What is the time and space complexity of common linked list operations (insertion, deletion, search, traversal)?

14. Traverse a singly linked list and print all elements.

15. Find the length of a linked list (iterative and recursive).

Complexity Basics

16. Explain Big-O, Big-Theta, and Big-Omega notation with simple examples.

17. What is the difference between time complexity and space complexity? Give an example where you'd trade one for the other.

MEDIUM LEVEL

Linked List

1. Reverse a singly linked list, both iteratively and recursively.

2. Delete a specific node from a linked list, given only the head and the value to delete.

3. Detect a cycle in a linked list (Floyd's cycle detection / tortoise-and-hare algorithm).

4. Find the middle element of a linked list in a single pass.

5. Merge two sorted linked lists into one sorted linked list.

Sorting & Searching

6. Implement Merge Sort from scratch and explain why it's O(n log n) in all cases.

7. Implement an optimized Sieve of Eratosthenes to print all primes in a range, and explain the complexity improvement over brute force.

8. Implement Binary Search (iterative and recursive) and explain the invariant that must hold at each step.

9. Given a rotated sorted array, find a target element in O(log n) time.

Arrays / Strings — Applied

10. Implement Kadane's Algorithm to find the maximum sum subarray. Explain why it works.

11. Given two arrays representing string1 and string2 characters, determine if string3 is a valid interleaving (order-preserving merge) of string1 and string2.

```
s1 = "aabcc", s2 = "dbbca", s3 = "aadbbcbcac"  // should be true
```

12. Given two strings s1 and s2, and a third string s3, verify if s3 is formed by combining s1 and s2 in order (interleaving check), e.g.:

```
String s1 = "abc";
String s2 = "def";
String s3 = "adbcfe";
// s3 formed as ad+bcef -> is this a valid interleave?
```

13. Given an array of strings, group anagrams together.

14. Find all pairs in an array that sum to a given target (not just one pair — return all unique pairs).

Recursion / Backtracking

15. Generate all subsets (the power set) of a given array.

16. Generate all permutations of a given string/array.

17. Solve the classic N-Queens problem (conceptually explain the backtracking approach).

Classic Puzzle / Aptitude-style

18. You have 4 bottles, one of which contains poison. You have a limited number of test strips/mice. What is the minimum number of testers needed to identify the poisoned bottle? Generalize the logic (binary encoding of bottle IDs).

19. Swap two variables without using a temporary variable — show at least 2 different approaches (arithmetic and XOR).

HARD LEVEL

Dynamic Programming

1. Given two words, word1 and word2, find the minimum number of operations (insert, delete, replace) required to convert word1 into word2 (Edit Distance / Levenshtein Distance).

2. Solve the 0/1 Knapsack problem and explain the recurrence relation and its optimized space complexity version.

3. Find the Longest Common Subsequence (LCS) between two strings.

4. Find the Longest Increasing Subsequence (LIS) in O(n log n).

5. Solve the Coin Change problem (minimum number of coins to make a given amount) and explain why a greedy approach fails for certain coin systems.

Trees & Graphs

6. Implement level-order (BFS) and in-order/pre-order/post-order (DFS) traversal of a binary tree.

7. Find the Lowest Common Ancestor (LCA) of two nodes in a Binary Search Tree, and then in a general Binary Tree.

8. Check whether a binary tree is height-balanced.

9. Implement BFS and DFS for a graph (adjacency list) and detect a cycle in a directed graph vs an undirected graph.

10. Implement Dijkstra's algorithm for shortest path in a weighted graph, and explain when Dijkstra fails (negative weights) and what algorithm you'd use instead (Bellman-Ford).

11. Find the number of islands in a 2D grid (connected components via DFS/BFS).

Advanced Array / Sliding Window

12. Fraud Detection problem: Given a stream of transactions {userId, amount, timestamp}, design an efficient algorithm (not brute force O(n^2)) to detect userIds with 3+ transactions within any 600-second window where amounts are within 10% of each other. Discuss time complexity.

```
const transactions = [
  { userId: 1, amount: 100, timestamp: 1690000100 },
  { userId: 2, amount: 200, timestamp: 1690000150 },
  { userId: 1, amount: 105, timestamp: 1690000200 },
  { userId: 1, amount: 98,  timestamp: 1690000250 },
  { userId: 2, amount: 210, timestamp: 1690000850 },
  { userId: 1, amount: 110, timestamp: 1690000500 },
  { userId: 3, amount: 300, timestamp: 1690000600 },
];
```

■ *Group transactions by userId → sort by timestamp per user → sliding window over each user's sorted transactions checking time span and amount similarity → overall O(n log n)*

13. Given an array, find the maximum sum of a subarray of size K using the sliding window technique (avoid the brute force O(n*k)).

14. Find the longest substring without repeating characters using the sliding window + hashmap technique.

15. Given an array of integers and a target, find if there exists a subarray with sum exactly equal to target (handle negative numbers using prefix sums + hashmap).

Design / Data Structure Implementation

16. Design and implement an LRU Cache with O(1) get and put operations (HashMap + Doubly Linked List).

17. Implement a Min-Heap / Max-Heap from scratch and use it to solve the 'Kth largest element in a stream' problem.

18. Implement a Trie (prefix tree) supporting insert, search, and startsWith operations.

19. Design a data structure that supports insert, delete, and getRandom in O(1) average time.

Complexity & Optimization Discussion

20. Given a brute-force O(n^2) solution to a problem (e.g. pair sum), walk through the optimization process to get to an O(n) or O(n log n) solution — explain your thought process, not just the final code.

21. Compare the time/space trade-offs between a recursive and an iterative (memoized/tabulated) solution to a DP problem.